ONLINE BOOKING

Elias Aschwanden, Jan Bretscher
elias.aschwanden@lernende.bbw.ch, jan.bretscher@lernende.bbw.ch

Inhalt

1 Planung 2
1.1 Appbeschreibung 2
1.2 Appaufteilung 2

1 Planung

1.1 Appbeschreibung

Mit dieser App können Benutzer ihre eigenen Angebote und Dienstleistungen erstellen und verwalten. Die Benutzer können einen Account erstellen und sich damit anmelden. Hat man das Passwort vergessen, wird ein wiederherstellungslink per Mail versendet. Die Authentifizierung wird mit jwt-Tokens organisiert. Nach dem Erstellen von einem Account kann man zwischen vier verschiedenen Sprachen (Deutsch, Englisch, Französisch und Italienisch) wählen. Der Hauptteil der Anwendung ist das Angebotssystem, in dem Mitglieder ihre Angebote in verschiedenen Kategorien erstellen und andere Angebote durchsuchen und buchen können. Man kann so Angebote von anderen Benutzern kaufen, und selbst Angebote erstellen.

1.2 Appaufteilung

App 1 (Frontend)

Sprache:

TypeScript (React)

Funktionen:

User authentication system (login, register, password reset)

Angebot Management (erstellen, bearbeiten, löschen)

Buchung Management (Angebote buchen)

Benutzerprofil Management

Suchfunktion für Angebote

Filterung nach Kategorien, Ersteller

Ort:

Web-based application

Persistenz:

LocalStorage für Session-Daten (JWT)

API-Kommunikation mit Backend für Kundendaten

Beschreibung:

Das Frontend ist eine responsive Webapp. Benutzer können sich registrieren, authentifizieren und ihr Profil verwalten. Die Hauptfunktionalität ist das Erstellen, Bearbeiten und Löschen von Angeboten sowie das Buchen vorhandener Angebote.

Begründungen:

TypeScript mit React um ein unkompliziertes UI zu erstellen

Responsive Design

App 2 (Backend)

Sprache:

JavaScript (Node.js)

MySQL

Funktionen:

REST API Endpoints für Authentication

Endpoints für Angebote

Endpoints für Buchungen

Benutzer-Management Endpoints

Profile Management Endpoints

Datenbankabfragen und -verbindung

Ort:

Web based Application

Persistenz:

Relationale Datenbank

Speicherung von Benutzerdaten, Angeboten, Buchungen, Authentifizierungstokens

Beschreibung:

Das Backend ist eine Node.js REST API, die als Schnittstelle zwischen Frontend und Datenbank fungiert. Es verwaltet alle Allgemeine-Logik, Authentifizierung und Datenverwaltung. Die API bietet sichere Endpoints mit JWT-Token-Authentifizierung.

Begründungen:

JavaScript/Node.js für übersichtliche Webapi

JWT für die Authentication der User

MySQL Datenbank für die Speicherung der Kunden und Angebotsdaten

JavaScript/Node.js für übersichtliche Webapi

JWT für die Authentication der User

MySQL Datenbank für die Speicherung der Kunden und Angebotsdaten

1.3 Arbeitsschritte

1. Zuerst haben wir auf Azure die Frontend und B ackend App Services aufgesetzt.
2. Danach haben wir auf NeonDB eine Datenbank aufgesetzt.
3. Danach haben wir die Datenbank, die wir machen wollten, designt.
4. Nun haben die geplante DB umgesetzt.
5. Als nächstes haben wir das Frontend und das Backend erstellt und programmiert.

2 Technische Details

2.1 Installation & Nutzung

2.1.1 Voraussetzungen

- **Node.js**: Version 18 oder höher
- **Datenbank**: PostgreSQL (NeonDB) gemäss `.env` Konfiguration

2.1.2 Backend (Server)

1. Navigieren Sie in das Backend-Verzeichnis: `cd booking-webapp_backend`
2. Installieren Sie die Abhängigkeiten: `npm install`
3. Erstellen Sie eine `.env` Datei basierend auf Ihren Datenbank-Credentials.
4. Starten Sie den Server: `npm start` Der Server läuft standardmäig auf `http://localhost:3040` (oder Port in env).

2.1.3 Frontend (Client)

1. Navigieren Sie in das Frontend-Verzeichnis: `cd booking-webapp_frontend`
2. Installieren Sie die Abhängigkeiten: `npm install`
3. Starten Sie den Entwicklungsserver: `npm run dev` Die App ist unter `http://localhost:3030` erreichbar.

2.2 Architektur

Die Anwendung folgt einer klassischen Client-Server-Architektur. Das Frontend (SPA) kommuniziert über eine REST-API mit dem Backend.

```
graph TD
User((User)) -->|HTTPS| Client[Frontend (React SPA)]
Client -->|JSON / REST API| Server[Backend (Node.js / Express)]
Server -->|SQL| DB[(Database MySQL/PostgreSQL)]
```

2.2.1 Komponenten

- **Frontend**: Übernimmt die Darstellung und Benutzerinteraktion. Nutzt LocalStorage für Session-Persistenz (JWT).
- **Backend**: Geschäftslogik, Validierung, Authentifizierung und Datenbankzugriff.
- **Datenbank**: Speichert persistente Daten wie User, Angebote (Offers) und Buchungen.

2.3 API Beschreibung

Die API ist REST-basiert und verwendet JSON für den Datenaustausch. Authentifizierung erfolgt, wo nötig, via Bearer Token (JWT).

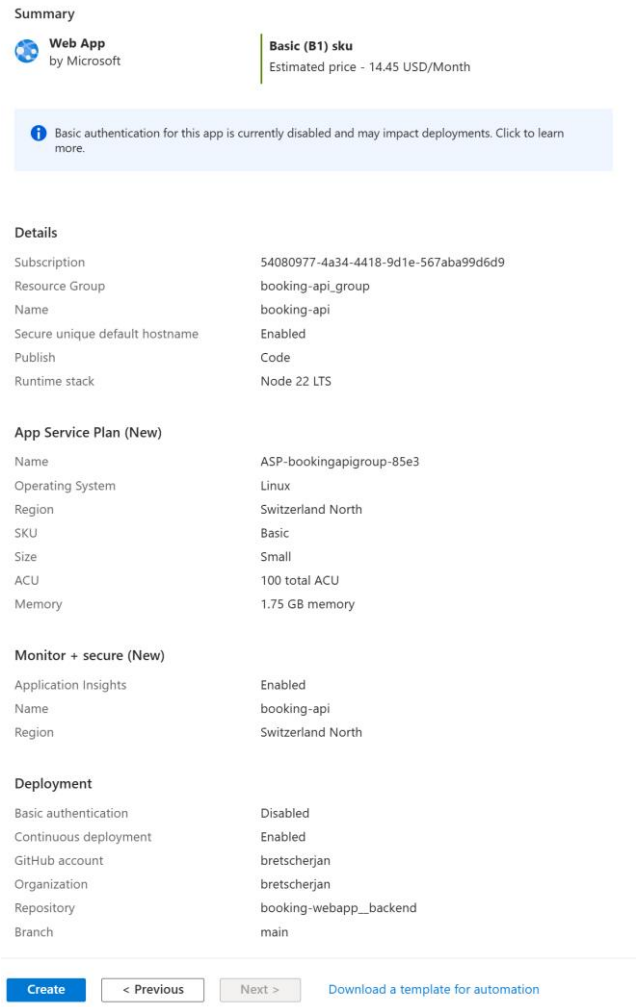
2.3.1 Endpoints (Übersicht)

- **Auth (`/auth`)** * `POST /register`: Neuen Benutzer registrieren * `POST /login`: Benutzer anmelden (erhält JWT)

- **User (`/user`)** * `GET /profile`: Eigenes Profil abrufen * `PUT /update`: Profil aktualisieren
- **Angebote (`/offers`)** * `GET /`: Alle Angebote suchen/filtern * `POST /`: Neues Angebot erstellen (Auth required) * `GET /:id`: Details eines Angebots * `DELETE /:id`: Angebot löschen
- **Buchungen (`/bookings`)** * `POST /`: Angebot buchen * `GET /my`: Eigene Buchungen ansehen
- **Admin (`/admin`)** * Verwaltungsfunktionen für Administratoren (User blockieren, etc.)

2.4 Deployment & Infrastruktur (Azure)

Das Deployment erfolgt vollautomatisiert (Frontend) bzw. skriptbasiert (Backend) auf Microsoft Azure. Wir nutzen eine **PaaS-Strategie** (Platform as a Service) mit Azure App Services.



2.4.1 Infrastructure as Code (IaC) - Bicep

Die Azure-Infrastruktur wird deklarativ mittels **Bicep** definiert. Dies ermöglicht reproduzierbare Deployments und Versionierung der Infrastruktur.

Resource Definition (`main.bicep`): Das Template definiert die `Microsoft.Web/sites` Ressource (App Service).

- **App Service Plan**: Referenziert via `existing` Keyword (Scope: `booking-frontend_group`), da dieser zentral verwaltet wird.
- **Configuration**: * `linuxFxVersion: 'NODE|22-lts'`: Erzwingt die Node.js 22 Runtime. * `appSettings`: Injeziert Umgebungsvariablen direkt in die App-Konfiguration. * `@secure() param`: Stellt sicher, dass die `DATABASE_URL` nicht in Logs auftaucht.

```
resource backendApp 'Microsoft.Web/sites@2022-09-01' = {
        name: backendAppName
        properties: {
                siteConfig: {
                        linuxFxVersion: 'NODE|22-lts'
                        appSettings: [
                                { name: 'DATABASE_URL', value: databaseUrl }
                        ]
                }
```

```
        }
}
```

Deployment Execution (`deploy.sh`): Das Shell-Skript abstrahiert die Azure CLI Kommandos.

1. **Environment Injection**: Lädt Variablen aus `.env` (`export $(grep -v '^#' .env | xargs)`).
2. **Validation**: Prüft ob kritische Variablen (z.B. `DATABASE_URL`) gesetzt sind.
3. **Provisioning**: Führt das Deployment auf Group-Level aus.

```
az deployment group create \
--resource-group $BACKEND_RG \
--template-file main.bicep \
--parameters databaseUrl="$DATABASE_URL"
```

2.4.2 CI/CD Automation (GitHub Actions)

Der CI/CD Prozess für das Frontend ist in `.github/workflows/deploy.yml` definiert.

Workflow Analyse (`build-and-deploy`):

- **Trigger**: `push` Event auf den `main` Branch.
- **Runner**: `ubuntu-latest`.
- **Key Actions**: 1. `actions/setup-node@v4`: Installiert Node.js 22. 2. `npm prune --production`: Entfernt `devDependencies` nach dem Build, um das Artefakt klein zu halten (Performance & Security). 3. `azure/webapps-deploy@v3`: Führt das eigentliche Deployment durch. Das `publish-profile` wird sicher aus den GitHub Secrets (`AZURE_WEBAPP_PUBLISH_PROFILE`) gelesen.

SPA Routing Workaround (`server.cjs`): Da Azure Linux App Services standardmässig nicht für Client-Side Routing (SPA) konfiguriert sind, wird die React-App von einem minimalen Express-Server ausgeliefert. Dieser Server fängt alle Requests ab, die **nicht** mit `/api` beginnen, und liefert die `index.html` zurück. Dies ermöglicht Deep-Linking in der React App.

```
// server.cjs
app.get(/^(?!\/api).+/, (req, res) => {
        res.sendFile(path.join(distPath, 'index.html'));
});
```

2.4.3 Verwendete GitHub Secrets

Damit die CI/CD-Pipeline und die Applikation funktionieren, müssen im GitHub Repository folgende Secrets hinterlegt sein:

Secret Name	Beschreibung	Verwendungszweck
`AZURE_WEBAPP_PUBLISH_PROFILE`	Publish Profile XML des Frontend App Services	Authentifizierung für Deployment (Frontend)

Secret Name	Beschreibung	Verwendungszweck
`VITE_API_BASE_URL`	URL des Backend-Services (z.B. `https://my-backend.azurewebsites.net/api`)	Wird beim Build in den React-Code eingebrannt

Hinweis Backend: Das Backend-Deployment via `deploy.sh` nutzt lokale Environment-Variablen oder Azure CLI Login-Kontext, benötigt daher keine GitHub Secrets für diesen Workflow, da es (aktuell) manuell/skriptbasiert ausgelöst wird.

2.4.4 Azure App Service Setup & Publish Profile

Falls eine manuelle Einrichtung des App Services notwendig ist (z.B. für das Frontend, falls nicht via Bicep erstellt), sind dies die Schritte:

1. App Service erstellen (Azure Portal):

1. Suche im Azure Portal nach „App Services" und klicke auf „Create" → „Web Service".
2. Wähle das passende Abonnement sowie die gewünschte Resource Group (z. B. booking-frontend_group) aus.
3. Vergib einen eindeutigen Namen (dieser wird Teil der URL, z. B. ihr-name.azurewebsites.net).
4. Wähle bei „Publish" die Option „Code".
5. Setze bei „Runtime stack" auf „Node 22 LTS".
6. Lege als „Operating System" „Linux" fest.
7. Wähle eine Region in deiner Nähe aus (z. B. Polen Central).
8. Klicke auf „Review + create" und anschließend auf „Create".

2. Publish Profile finden: Um GitHub Actions das Deployment zu erlauben, wird das **Publish Profile** benötigt.

1. **Navigiere** im Azure Portal zum gewünschten App Service.
2. **Klicke** auf der **Übersicht** (Overview) in der oberen Menüleiste auf **Veröffentlichungsprofil abrufen** (Get publish profile).
3. **Speichere** die .publishsettings-Datei auf deinem Rechner.
4. **Öffne** die Datei in einem Texteditor (z. B. Notepad++ oder VS Code).
5. **Kopiere** den gesamten XML-Inhalt in die Zwischenablage.

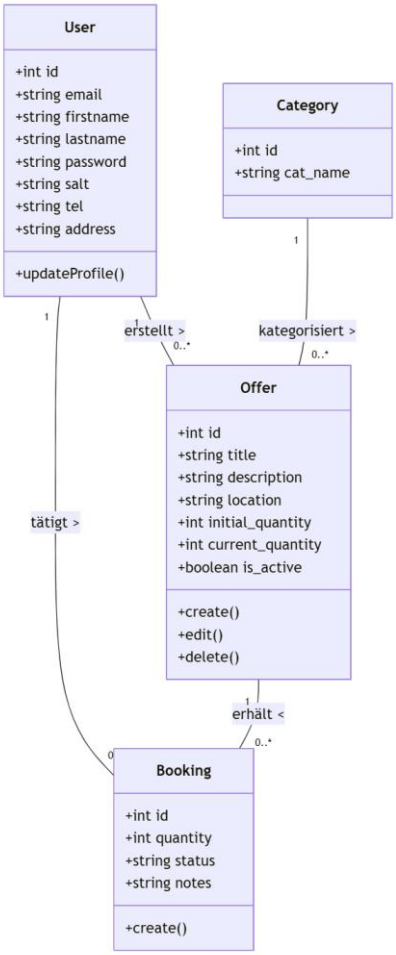
3. Secret in GitHub hinterlegen:

1. **Öffne** dein GitHub Repository und wechsle zum Tab **Settings**.
2. **Wähle** im linken Menü **Secrets and variables** > **Actions**.
3. **Klicke** auf die Schaltfläche **New repository secret**.
4. **Gib** als Name `AZURE_WEBAPP_PUBLISH_PROFILE` ein.
5. **Füge** den kopierten XML-Inhalt in das Feld **Value** ein.
6. **Bestätige** den Vorgang mit einem Klick auf **Add secret**.

2.5 Projekt-Logik & Datenmodell

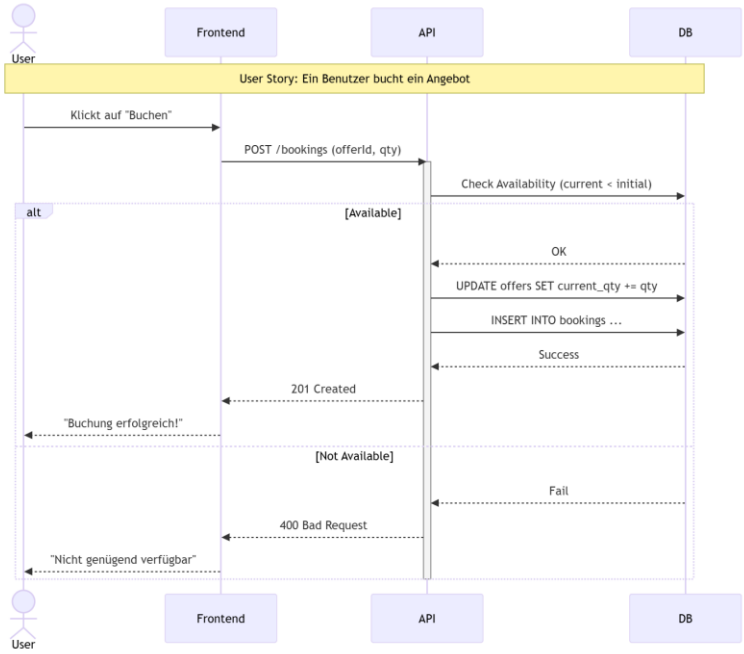
2.5.1 Klassen / Datenbank Design

Das Datenmodell basiert auf relationalen Tabellen in einer Neon PostgreSQL Datenbank. Hier ist das Schema als Klassendiagramm visualisiert, welches die Entitäten und ihre Beziehungen darstellt.



2.5.2 User Flows (User Stories)

Die Hauptabläufe der Anwendung unterscheiden zwischen einfachen Nutzern und der Interaktion mit Angeboten.



2.6 Technische Besonderheiten & Sicherheit

2.6.1 Datenbank-Verbindung

Wir verwenden **NeonDB**, eine serverlose PostgreSQL-Datenbank.

- **Treiber**: `@neondatabase/serverless` (Optimiert für Serverless/Edge Environments).
- **Connection**: HTTP-basiert (statt klassischem TCP), was in serverlosen Umgebungen stabiler und schneller skaliert.
- **Query-Wrapper**: Da wir SQL-Queries teilweise MySQL-Syntax (`?`) nutzen, haben wir einen Helper geschrieben, der diese on-the-fly in Postgres-Syntax (`$1, $2`) umwandelt.

2.6.2 9.2 Sicherheit

- **Passwörter**: Werden **nie** im Klartext gespeichert. Wir nutzen `bcrypt` mit einem zufällig generierten **Salt** pro User. * Workflow: `Hash = bcrypt(UserPassword + Salt)`

- **Session Management**: JSON Web Tokens (**JWT**). * Beim Login erhält der User ein Token. * Jeder geschützte Request muss dieses Token im `Authorization`-Header senden. * Das Backend verifiziert die Signatur des Tokens bei jedem Request.
- **Environment Variables**: Sensitive Daten (DB-URL, JWT-Secret) sind ausschliesslich in `.env` Dateien (lokal) oder in den Azure App Service Settings (Production) gespeichert und nicht im Code.